1 Lesson 03-11-26

During the last lesson we talked about *branch prediction tables* used for *dynamic branch prediction*.

We move on to other schemas now.

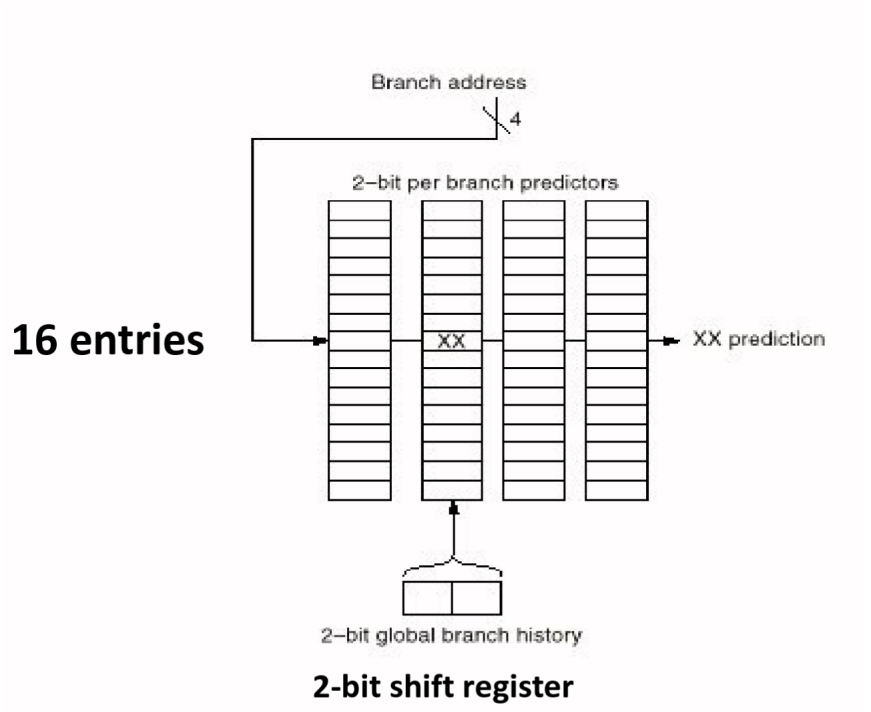
- **Correlating predictors** (also known as *Bilevel* predictors): this approach is based on the dependencies between the results of last branches. What we mean is that the behavior of a branch is strongly dependent on the result of the previous ones.

  (m, n) predictors use the behavior of the last m branches to choose from 2^m branch predictors, each of which is a n-bit predictor. The hardware required for implementing this scheme is very simple:

  - The history of the most recent m branches is recorded in an m-bit shift register, where each bit records whether the branch was taken or not;
  - The branch-prediction buffer is indexed using a concatenation of the low-order bits from the branch address with the m low-order history bits.

  Let's see a few examples:

  - $(1, 1)$ predictor: in this case, $m = 1$ and $n = 1$. Each branch is associated with 2 predictors of one bit: one reporting the prediction in the case the previous branch was taken, one with the prediction in the case the previous branch was not taken;
  - $(2, 2)$ predictor: same as above, but with $m = 2$ and $n = 2$. Each branch is associated with 4 predictors of 2 bits.



1.0.1 Branch-Target Buffer

The **BTB** (*Branch Target Buffer*) reduces an additional delay introduced by branch instructions: the fact that we may not know the delay target.

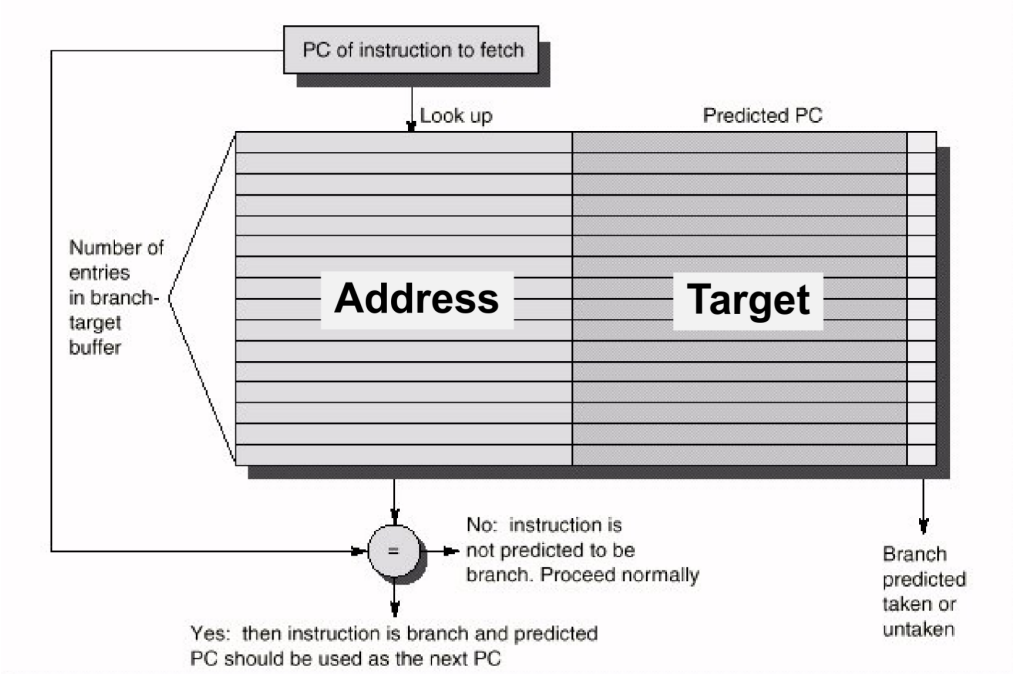
Reducing the negative effects of control dependencies requires knowing as soon as possible both:

- Whether the branch has to be taken or not;
- The new value of the PC (if the branch is assumed to be taken).

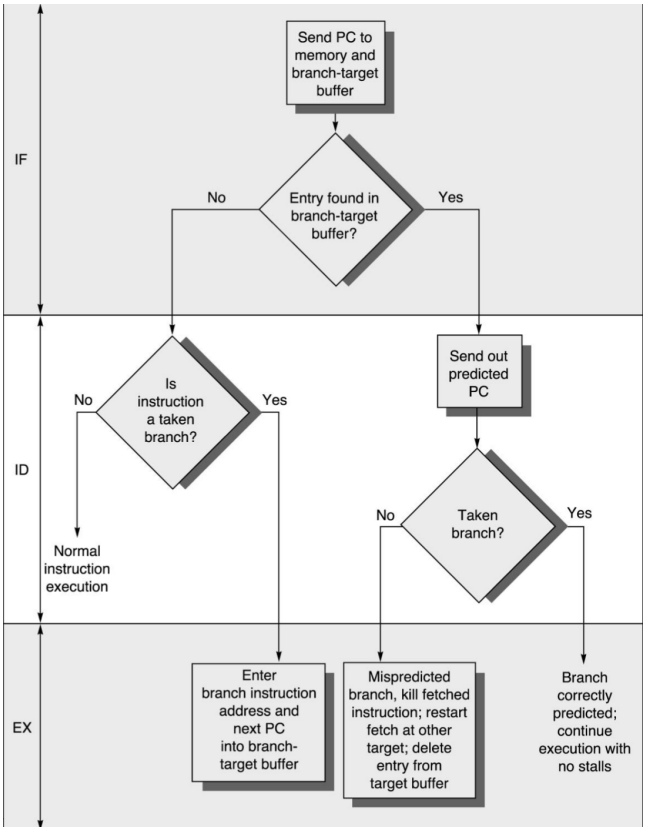
This later issue is faced by introducing a *branch-target buffer* (or cache). Each entry of the branch-target buffer contains:

- The address of the considered branch;
- The target value to be loaded in the PC.

Using the branch-target buffer, the PC is loaded with the new value at the end of the IF stage, i.e., even before the branch instruction is decoded.



We have a flowchart that shows how we interact with the BTB based on the execution phase of a branch instruction we find ourselves in:


IF
ID
EX
Send PC to memory and branch-target buffer
Entry found in branch-target buffer?
No
Yes
Is instruction a taken branch?
No
Yes
Normal instruction execution
Send out predicted PC
Taken branch?
No
Yes
Enter branch instruction address and next PC into branch-target buffer
Mispredicted branch, kill fetched instruction; restart fetch at other target; delete entry from target buffer
Branch correctly predicted; continue execution with no stalls